MULTIWAY POWERSORT

Ben Smith
William Cawley Gelling, Markus E. Nebel, and Sebastian Wild

University of Liverpool

Monday 23rd January, 2023

Timsort and Powersort

- Tim Peters introduced Timsort in 2002
  - highly engineered version of mergesort
  - comparison based, internal, and stable
  - multiply adaptive:
    - galloping merges
    - run detection on the fly

Timsort and Powersort

- Tim Peters introduced Timsort in 2002
  - highly engineered version of mergesort
  - comparison based, internal, and stable
  - multiply adaptive:
    - galloping merges
    - run detection on the fly



Timsort and Powersort

- Tim Peters introduced Timsort in 2002
  - highly engineered version of mergesort
  - comparison based, internal, and stable
  - multiply adaptive:
    - galloping merges
    - run detection on the fly
- Timsort's merge policy is less than optimal:
  - 1.5 times the optimal merge cost in the worst case (Buss and Knop 2018)
  - Hard to analyze
  - Prone to algorithmic bugs



Timsort and Powersort

- Tim Peters introduced Timsort in 2002
  - highly engineered version of mergesort
  - comparison based, internal, and stable
  - multiply adaptive:
    - galloping merges
    - run detection on the fly
- Timsort's merge policy is less than optimal:
  - 1.5 times the optimal merge cost in the worst case (Buss and Knop 2018)
  - Hard to analyze
  - Prone to algorithmic bugs

- Munro and Wild introduced Powersort in 2018
  - Merge policy update – retains most of Timsort
  - Optimal merge cost (up to lower order terms)
  - Built from first principles
  - Easy to analyze



Timsort and Powersort

- Tim Peters introduced Timsort in 2002
  - highly engineered version of mergesort
  - comparison based, internal, and stable
  - multiply adaptive:
    - galloping merges
    - run detection on the fly
- Timsort's merge policy is less than optimal:
  - 1.5 times the optimal merge cost in the worst case (Buss and Knop 2018)
  - Hard to analyze
  - Prone to algorithmic bugs

- Munro and Wild introduced Powersort in 2018
  - Merge policy update – retains most of Timsort
  - Optimal merge cost (up to lower order terms)
  - Built from first principles
  - Easy to analyze



Merge Cost and Merge Order

- Merge cost: An abstract cost measure introduced by Buss and Knop (2018)
- For a single operation merging k runs, $r_0, \ldots, r_{k-1}$, the merge cost is given by:

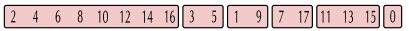
  $M = |r_0| + \ldots + |r_{k-1}|$

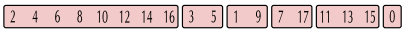
Merge Cost and Merge Order

- Merge cost: An abstract cost measure introduced by Buss and Knop (2018)
- For a single operation merging k runs, $r_0, \ldots, r_{k-1}$, the merge cost is given by:

  $M = |r_0| + \ldots + |r_{k-1}|$

Merge order affects total merge cost (Example with $k = 2$)



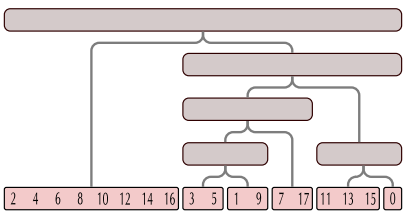


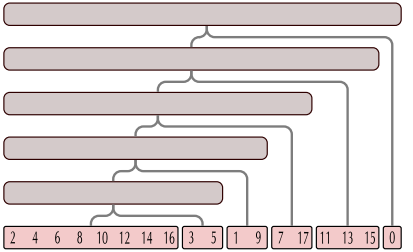
Merge Cost and Merge Order

- Merge cost: An abstract cost measure introduced by Buss and Knop (2018)
- For a single operation merging k runs, $r_0, \ldots, r_{k-1}$, the merge cost is given by:

  $M = |r_0| + \ldots + |r_{k-1}|$

Merge order affects total merge cost (Example with $k = 2$)



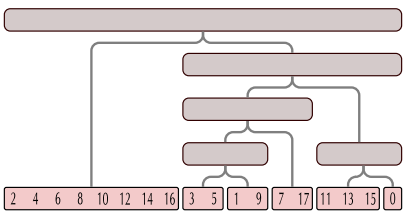


Merge Cost and Merge Order

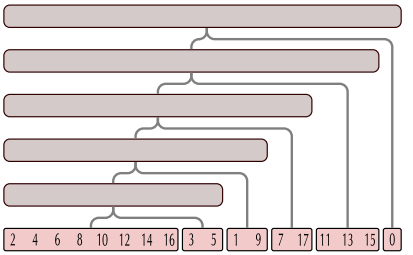
- Merge cost: An abstract cost measure introduced by Buss and Knop (2018)
- For a single operation merging k runs, $r_0, \ldots, r_{k-1}$, the merge cost is given by:

  $M = |r_0| + \ldots + |r_{k-1}|$

Merge order affects total merge cost (Example with $k = 2$)



merge costs: **42**



merge costs: **71**

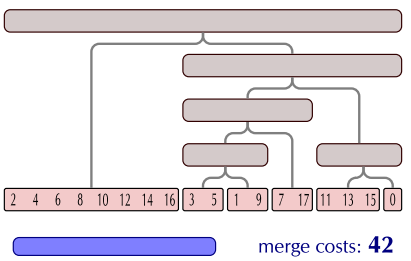
Merge Cost and Merge Order

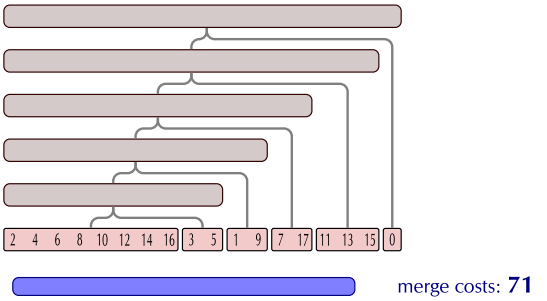
- Merge cost: An abstract cost measure introduced by Buss and Knop (2018)
- For a single operation merging k runs, $r_0, \ldots, r_{k-1}$, the merge cost is given by:

  $M = |r_0| + \ldots + |r_{k-1}|$

- An optimal merge tree minimizes total merge cost
- Goal: A nearly optimal merge tree with low overhead costs

Merge order affects total merge cost (Example with $k = 2$)

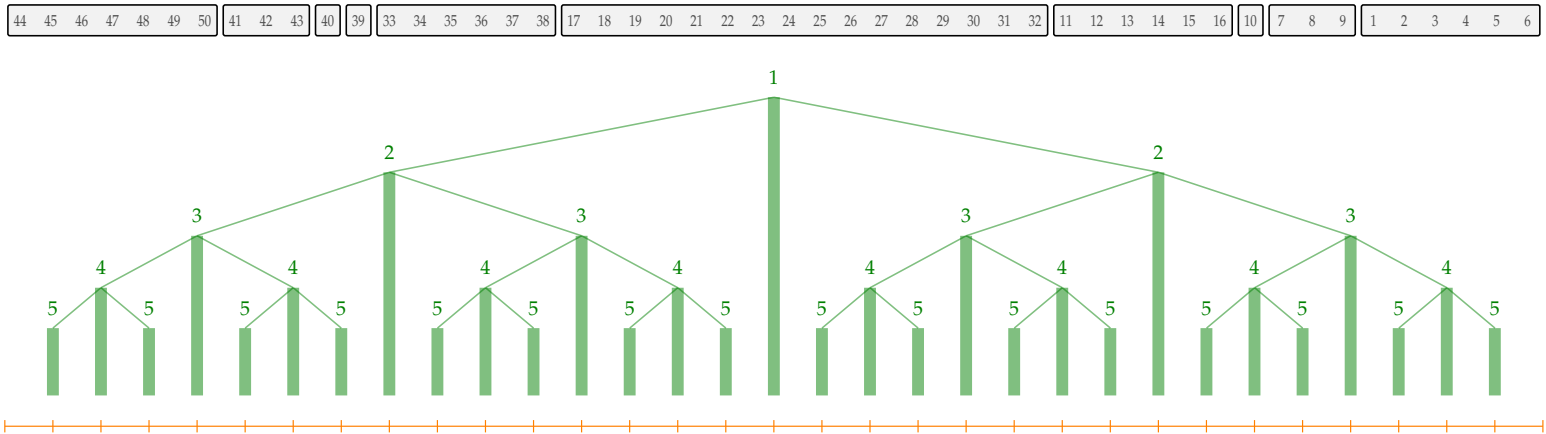




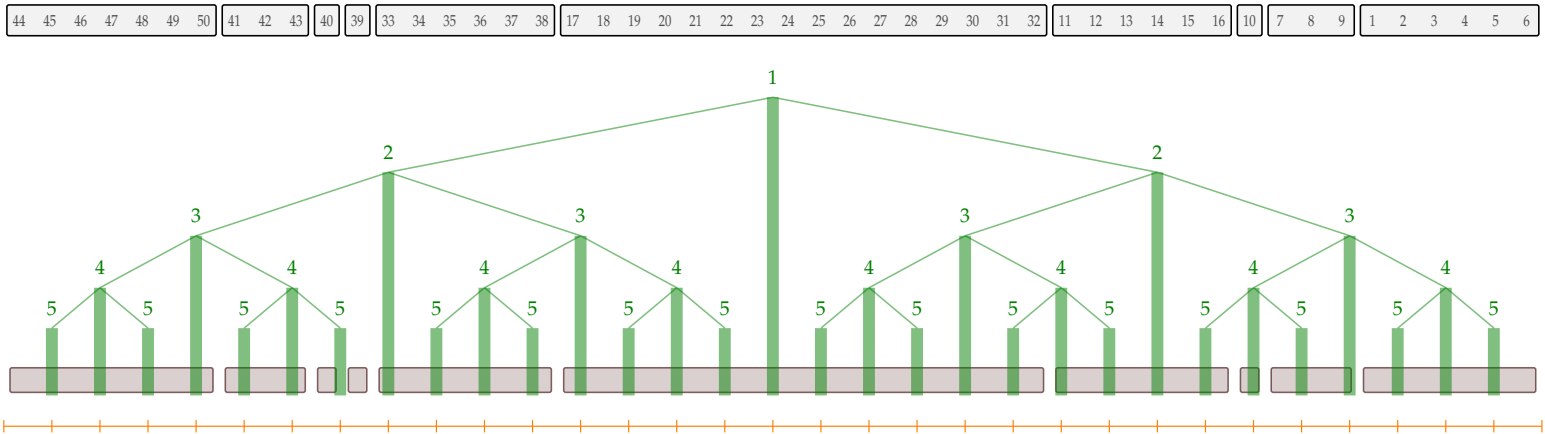
(2-way) Powers and the virtual perfectly balanced binary tree



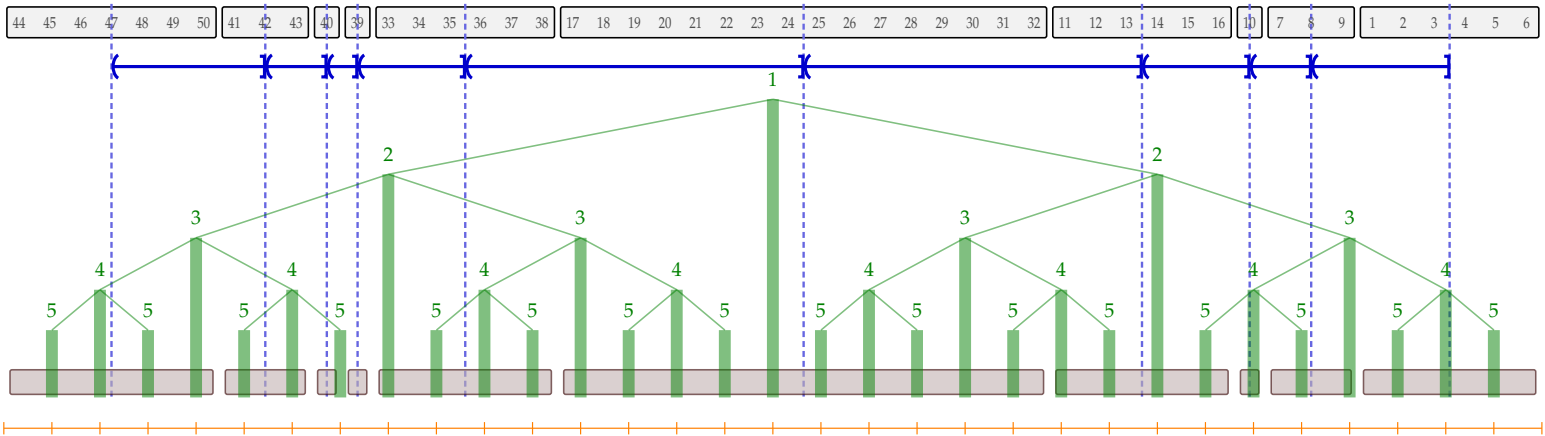
(2-way) Powers and the virtual perfectly balanced binary tree



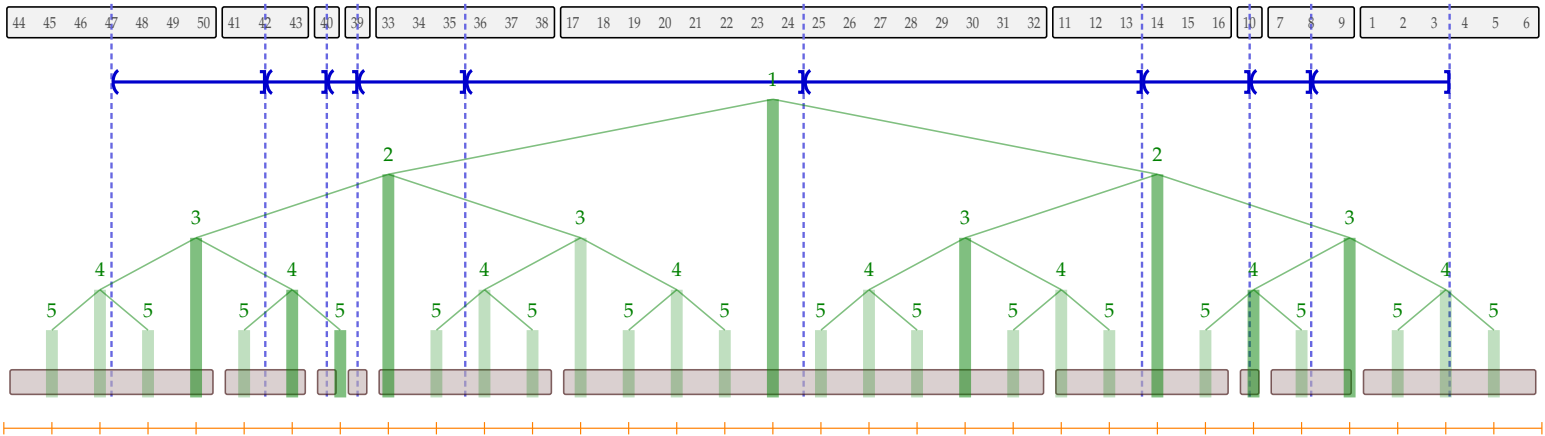
(2-way) Powers and the virtual perfectly balanced binary tree



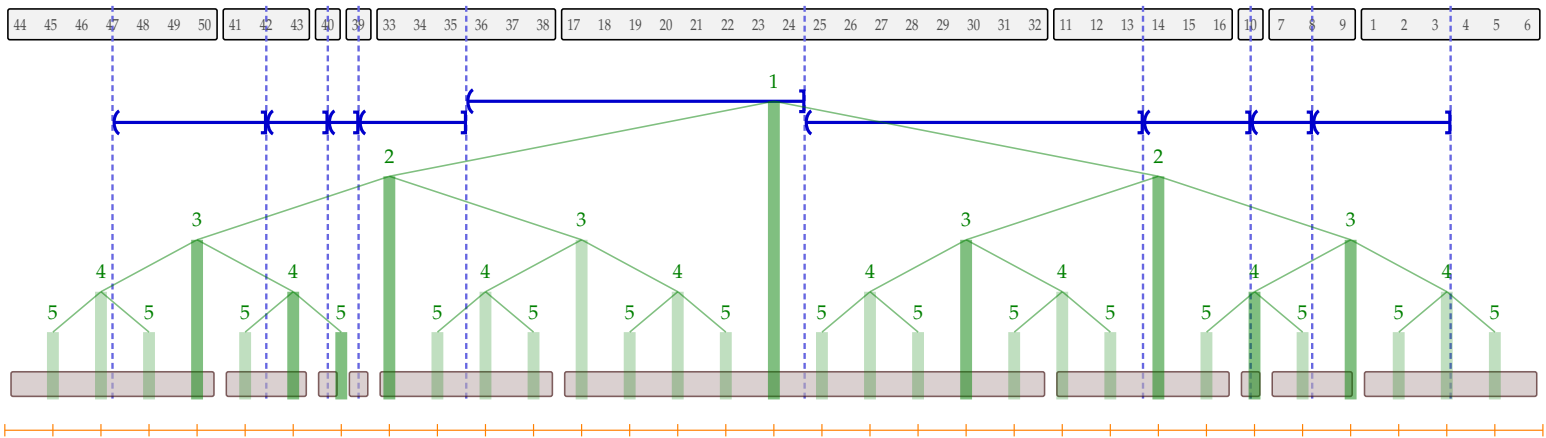
(2-way) Powers and the virtual perfectly balanced binary tree



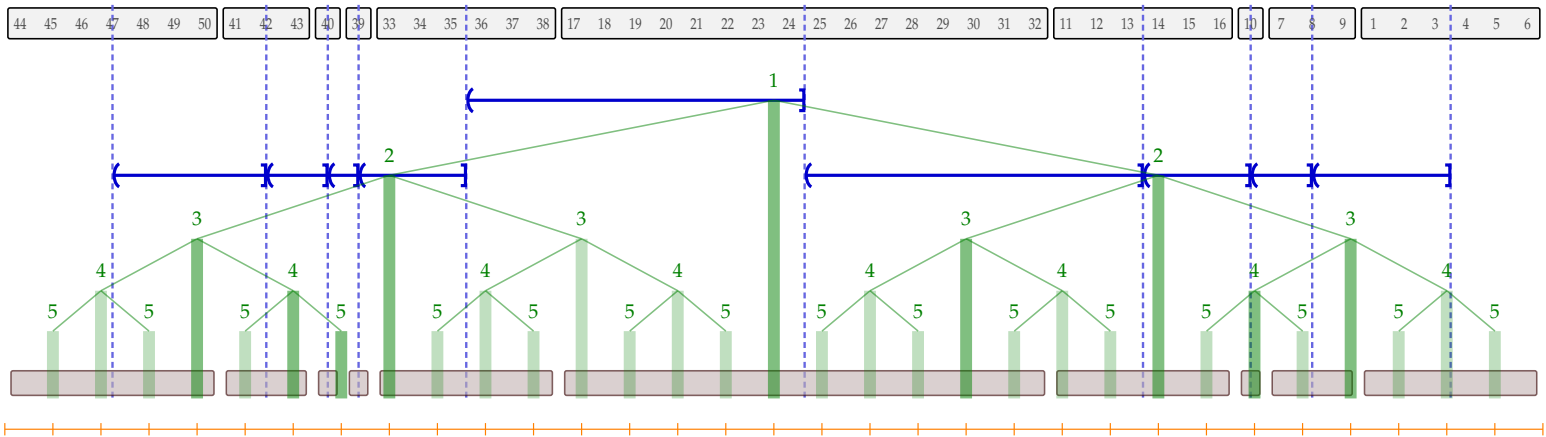
(2-way) Powers and the virtual perfectly balanced binary tree



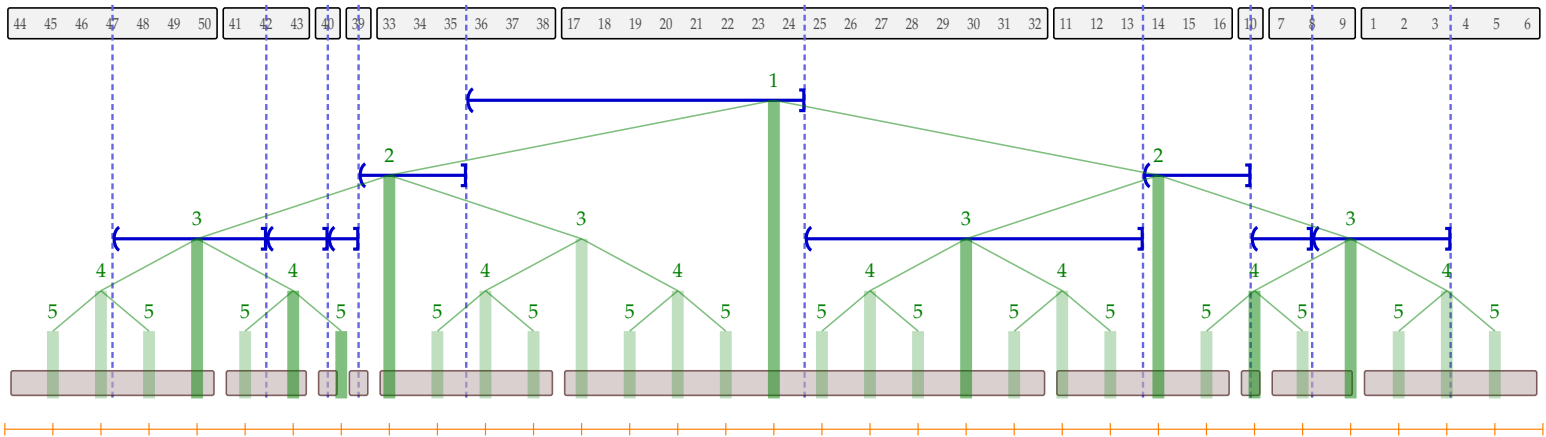
(2-way) Powers and the virtual perfectly balanced binary tree



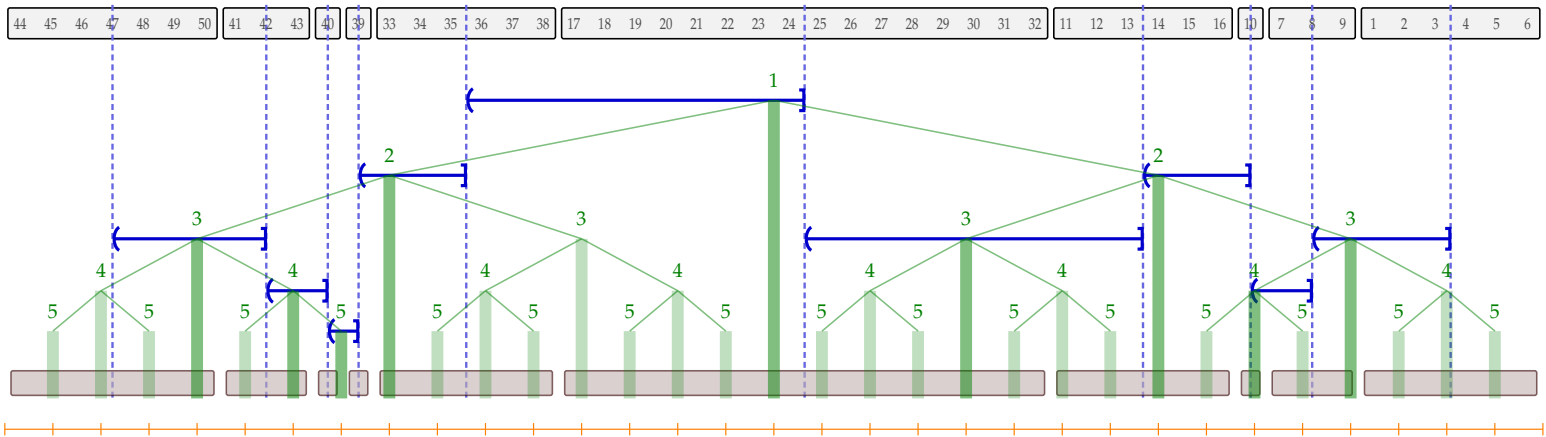
(2-way) Powers and the virtual perfectly balanced binary tree



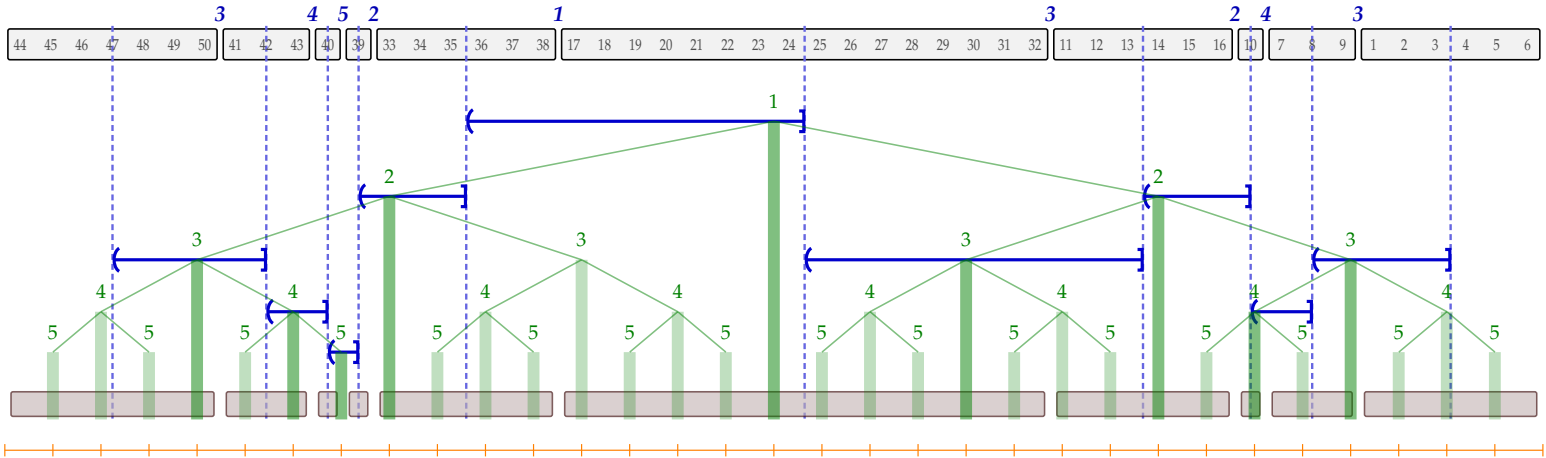
(2-way) Powers and the virtual perfectly balanced binary tree



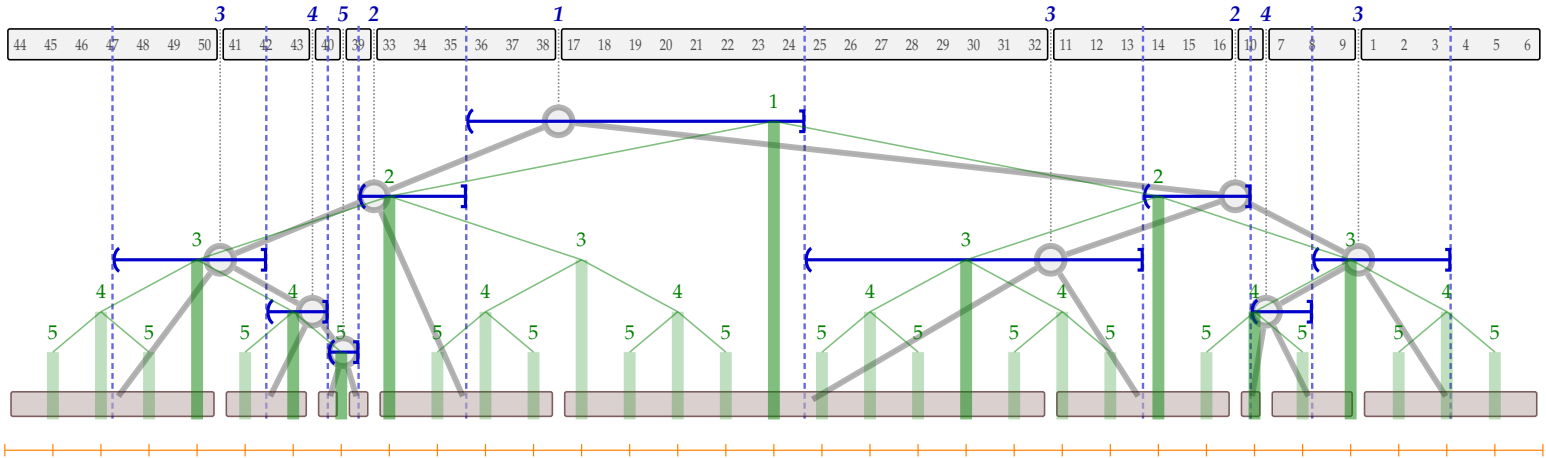
(2-way) Powers and the virtual perfectly balanced binary tree



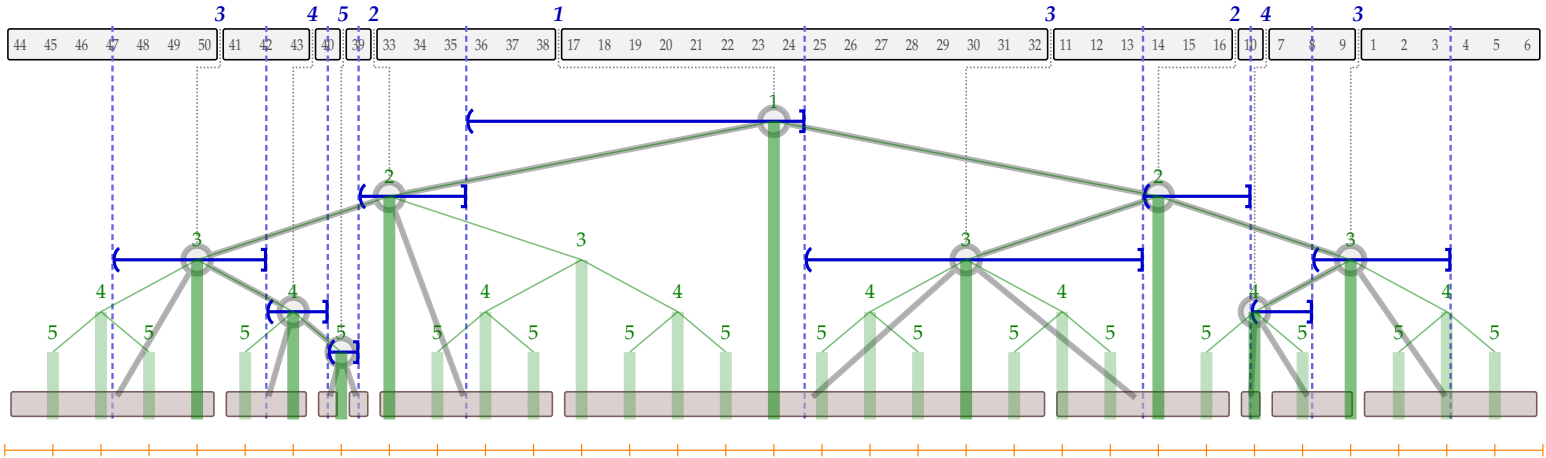
(2-way) Powers and the virtual perfectly balanced binary tree



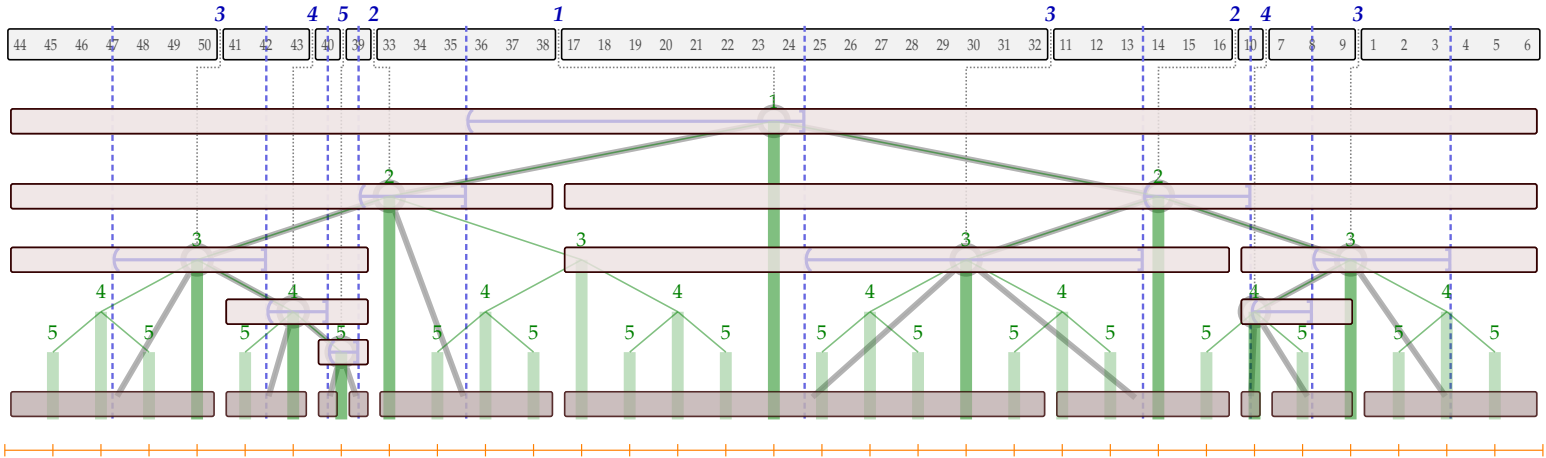
(2-way) Powers and the virtual perfectly balanced binary tree



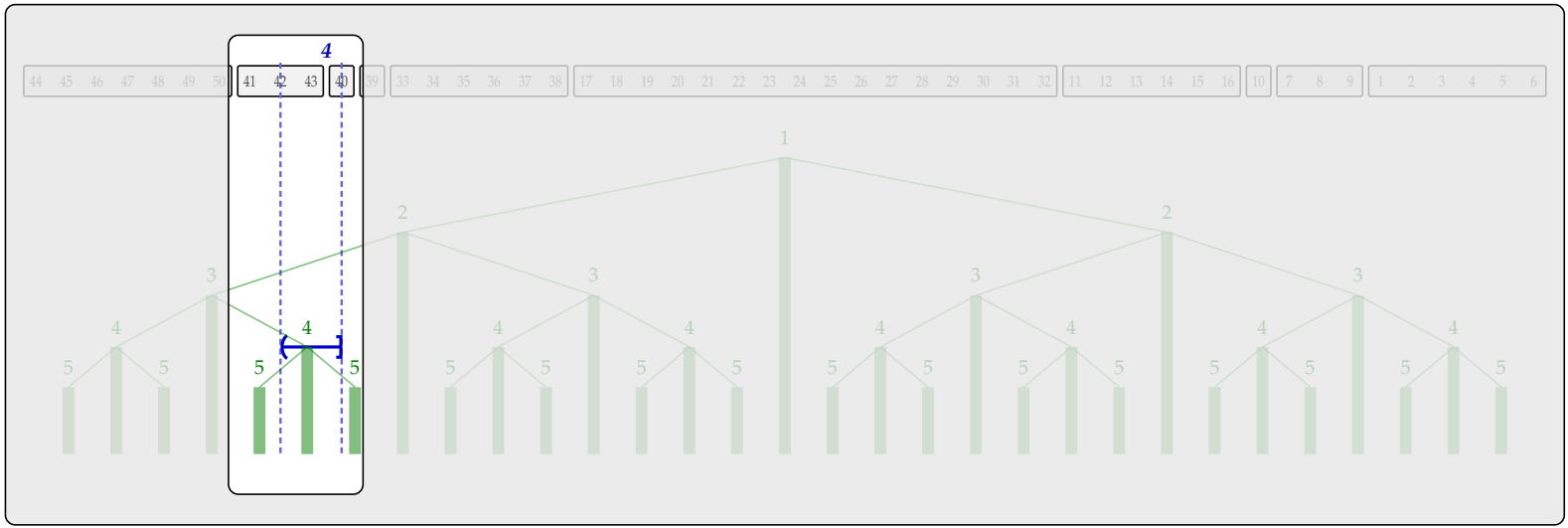
(2-way) Powers and the virtual perfectly balanced binary tree



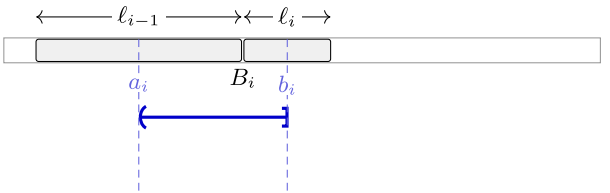
(2-way) Powers and the virtual perfectly balanced binary tree



Powers are local



Calculating merge powers



- For run lengths $L_0, \ldots, L_{r-1}$, first set $l_i = \frac{L_i}{n}$, then:
  - Define a_i, the normalized centre of the $(i-1)^{st}$ run:

$$a_i = \sum_{j=0}^{i-1} l_j - \frac{1}{2} l_{i-1},$$

  - Define b_i, the normalized centre of ith run:

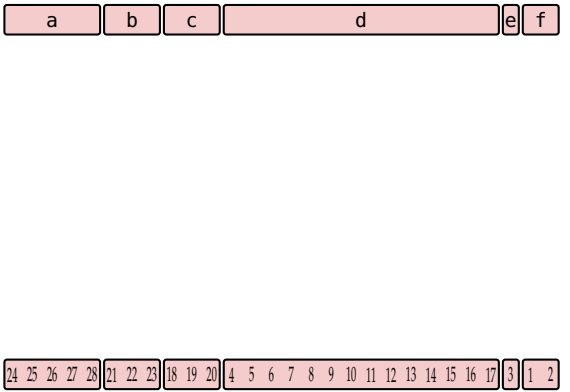
$$b_i = \sum_{j=0}^{i-1} l_j + \frac{1}{2} l_i$$

  - Hence, define the power of the boundary between these runs, P_i^k:

$$P_i^k = \min\{p \in \mathbb{N} : \lfloor a_i \cdot k^p \rfloor < \lfloor b_i \cdot k^p \rfloor\}$$

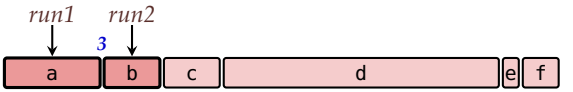
2-way Powersort

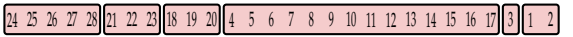
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



2-way Powersort

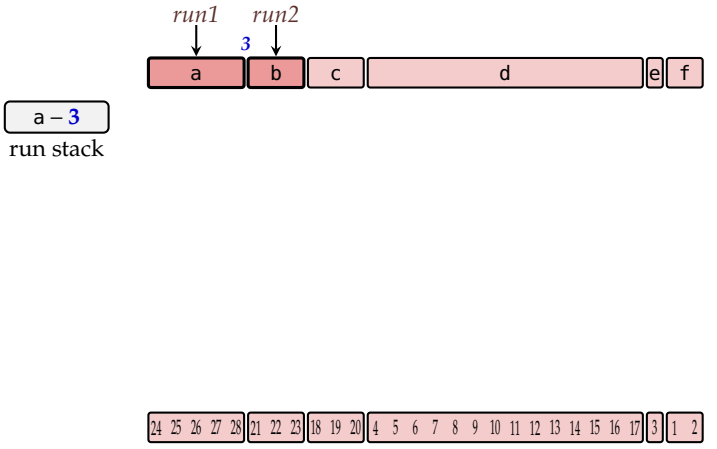
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```





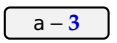
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

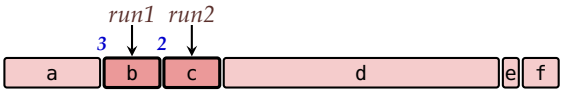


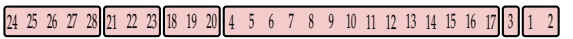
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



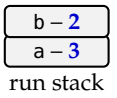
run stack

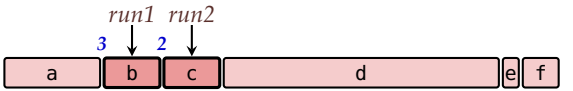


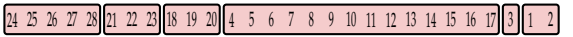


2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

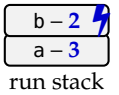


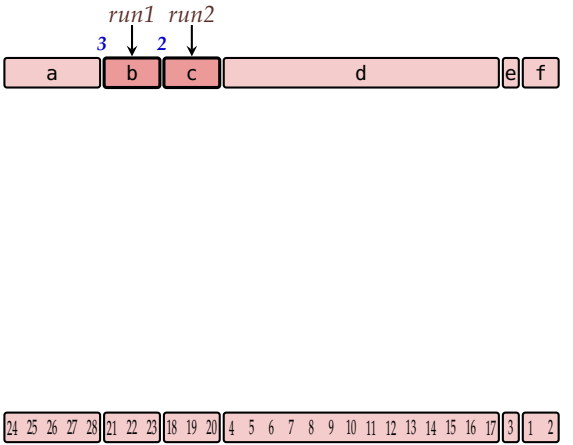




2-way Powersort

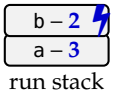
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

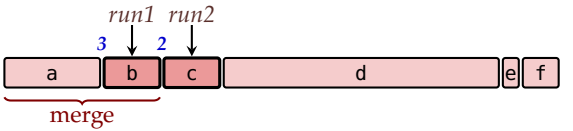


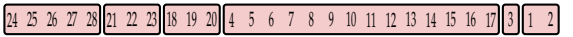


2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

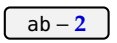




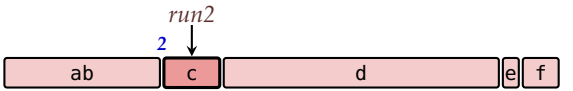


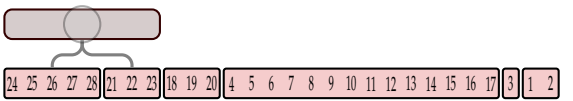
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



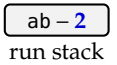
run stack

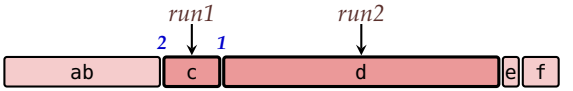


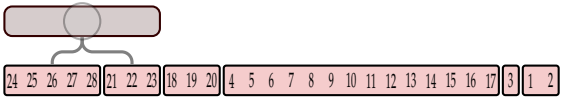


2-WAY POWERSORT

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

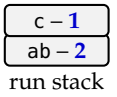


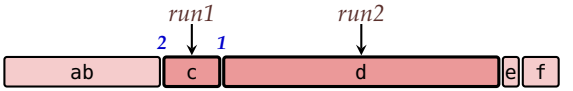


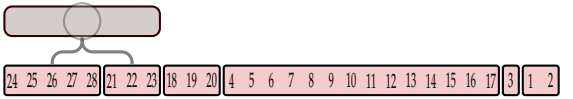


2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

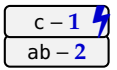






2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



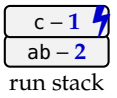
run stack

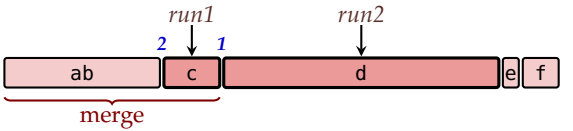


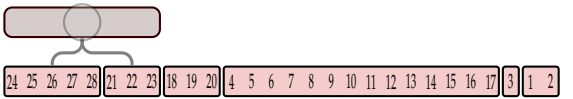


2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

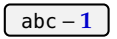






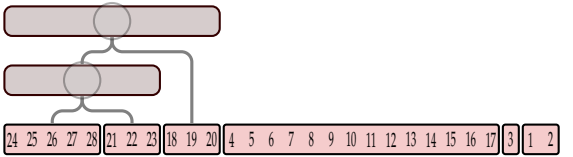
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



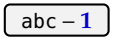
run stack





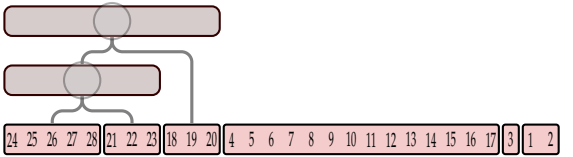
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



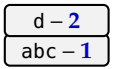
run stack





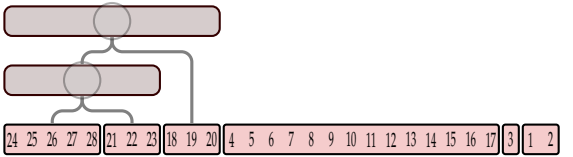
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



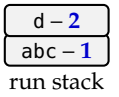
run stack

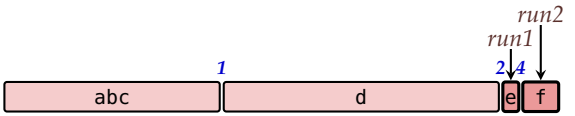


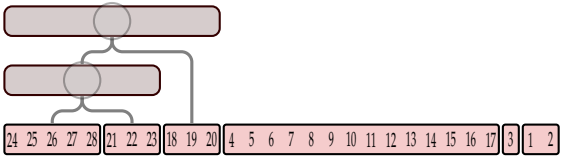


2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

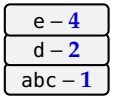




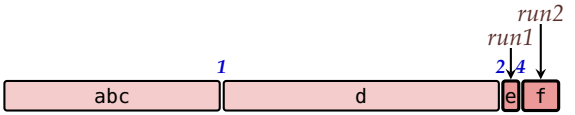


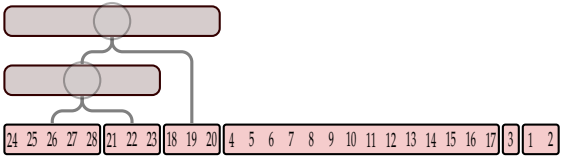
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



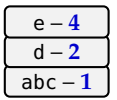
run stack



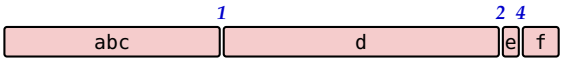


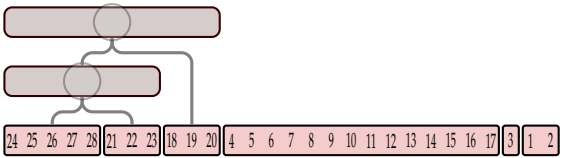
2-way Powersort

```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



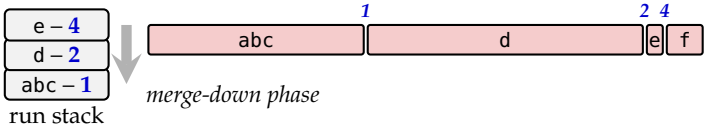
run stack

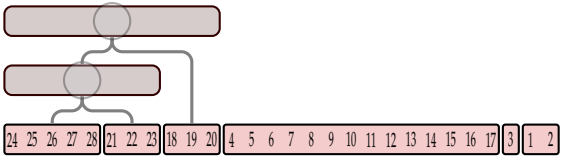




2-way Powersort

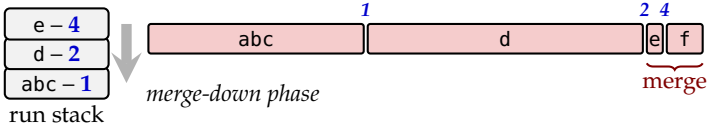
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

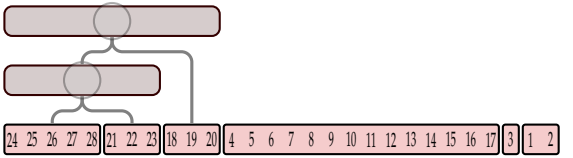




2-way Powersort

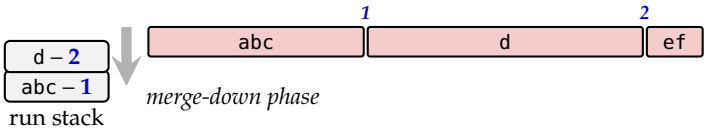
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

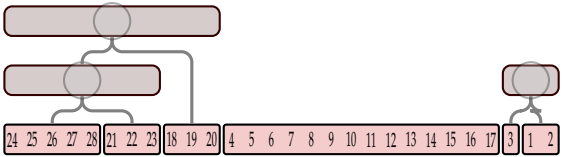




2-way Powersort

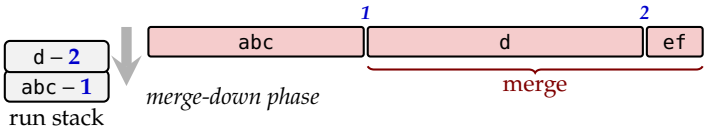
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```

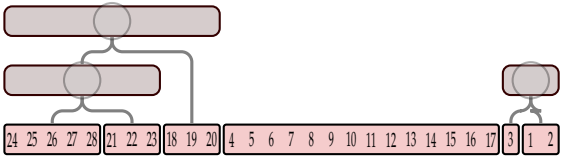




2-way Powersort

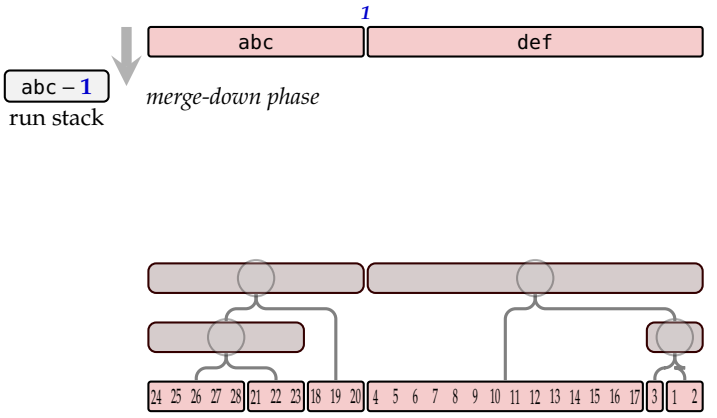
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```





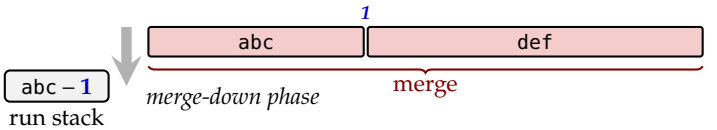
2-way Powersort

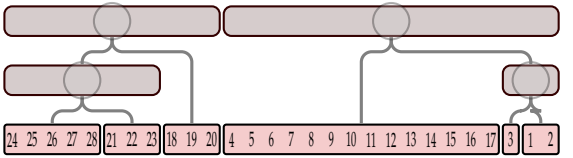
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



2-way Powersort

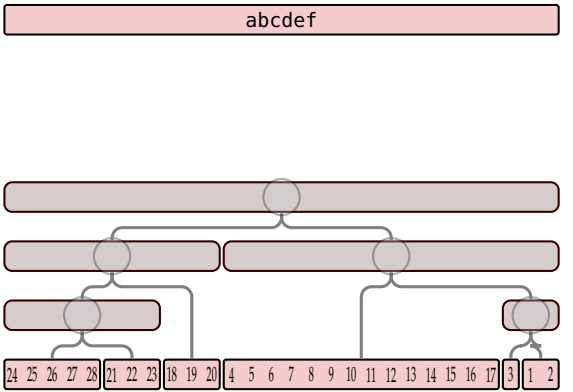
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```





2-way Powersort

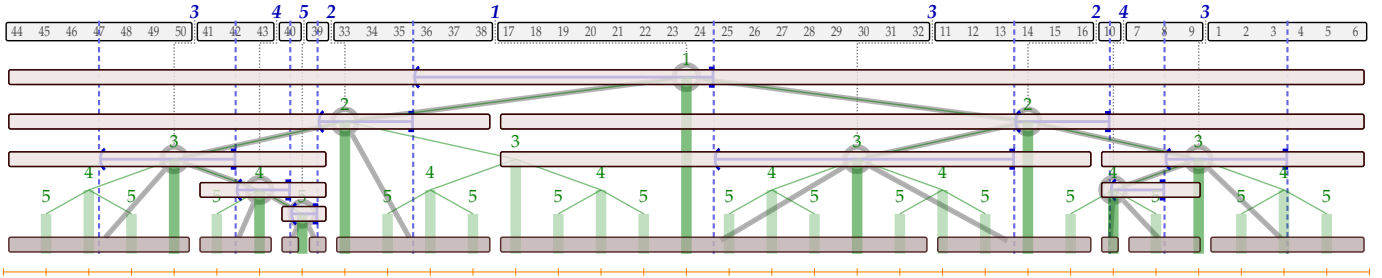
```
1  procedure Powersort(A[0..n))
2      i := 0; runs := new Stack()
3      j := ExtendRunRight(A, i)
4      runs.push(i, j); i := j
5      while i < n
6          j := ExtendRunRight(A, i)
7          p := power(runs.top(), (i, j), n)
8          while p ≤ topmost power
9              merge topmost 2 runs
10         runs.push(i, j); i := j
11     while runs.size() ≥ 2
12         merge topmost 2 runs
```



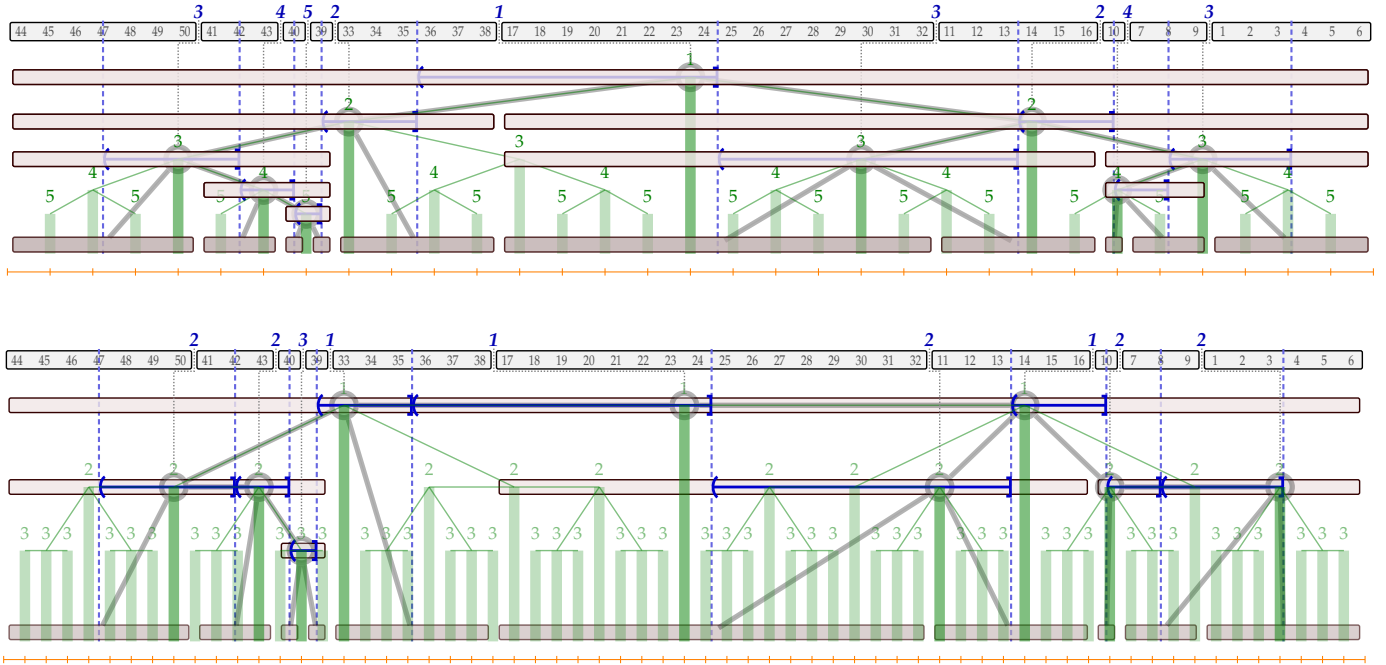
WHY MERGE MULTI-WAY?

- Memory transfer costs have become a tighter bottleneck than Processor costs in various sorting algorithms (Kushagra et.al., 2014)
  - But it's a close run thing!
- To reduce these costs, take inspiration from external memory techniques/algorithms
  - Specifically, ***k*-way merging**
- Tuning is important:
  - Low k reduces Processor costs and is easy to implement
  - High k reduces Memory costs (I/Os) but is harder to implement
  - Optimal value depends on data type, hardware, language,...

4 WAY MERGING



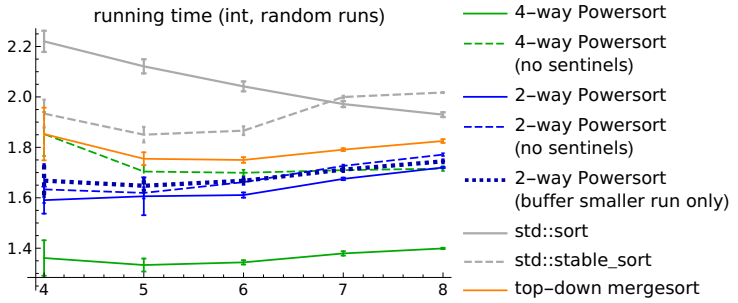
4 WAY MERGING



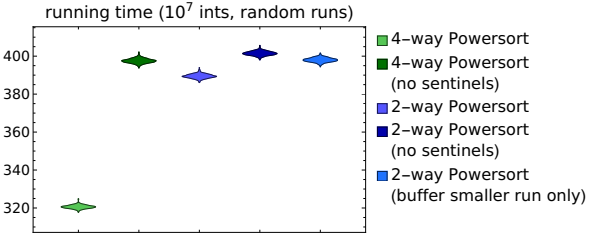
MULTIWAY MERGING

- New possibility: can now have multiple merges on the stack with the same power
- Handle but using a multi-way merge
- Specifically, a Winner tree, using sentinels wherever possible

RESULTS 1: RUNS OF EXPECTED LENGTH $\sqrt{n}$, INTS

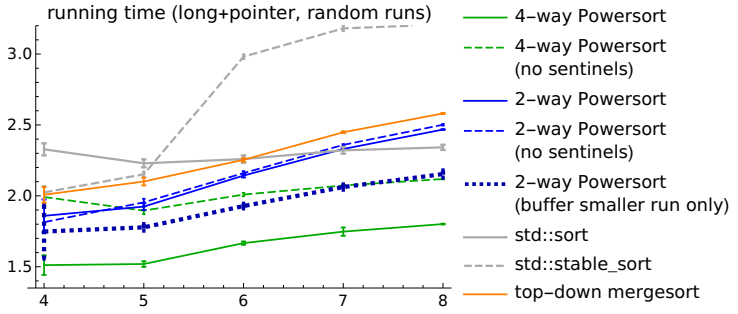


- Normalized running times against $\log_{10}(n)$.

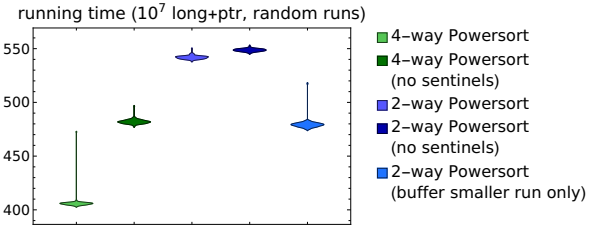


- Distributions of running time in ms for the same data

Results 2: Runs of expected length $\sqrt{n}$, long + pointer

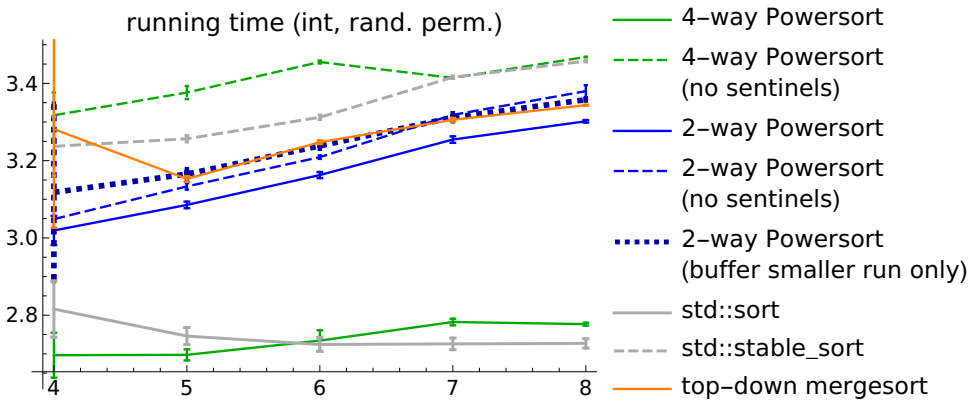


- Normalized running times against $\log_{10}(n)$.



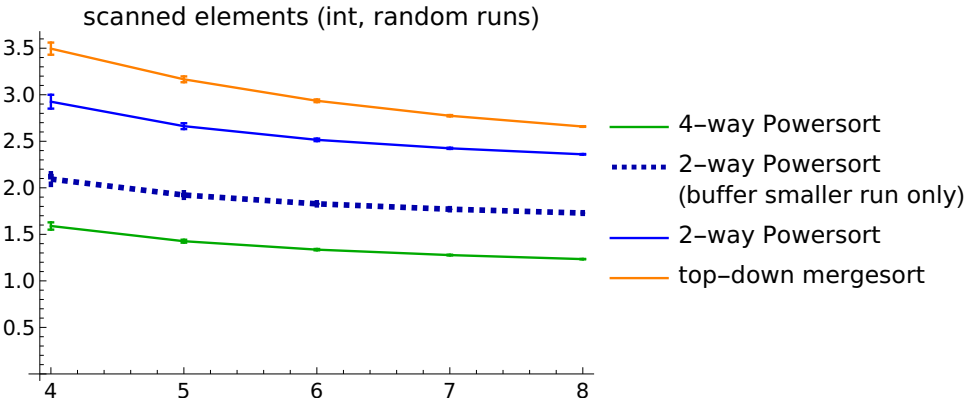
- Distributions of running time in ms for the same data

RESULTS 3: RANDOM PERMUTATIONS, INTS



- Normalized (time / $n \lg n$) running times against $\log_{10}(n)$.
- note that `std::sort` in C++ is intro sort – not a stable method

Results 4: Runs of expected length $\sqrt{n}$, ints



- Normalized merge cost against $\log_{10}(n)$.

Conclusion

- Findings
  - 4-way Powersort can yield significant performance improvements
  - This can be explained by its reduced merge cost
  - Some of these results rely on the use of sentinels

Conclusion

- Findings
  - 4-way Powersort can yield significant performance improvements
  - This can be explained by its reduced merge cost
  - Some of these results rely on the use of sentinels
- Further Work
  - Fast multi-way merging without simple sentinels
  - Study multi-way powersort in a wider technological context
  - Compare multi-way powersort with more competitors, specifically multiway quicksort variants.

Conclusion

- Findings
  - 4-way Powersort can yield significant performance improvements
  - This can be explained by its reduced merge cost
  - Some of these results rely on the use of sentinels
- Further Work
  - Fast multi-way merging without simple sentinels
  - Study multi-way powersort in a wider technological context
  - Compare multi-way powersort with more competitors, specifically multiway quicksort variants.
- Recommendations
  - Users of Timsort should seriously consider the Powersort merge policy
  - Users of Timsort and 2-way powersort may wish to consider multi-way Powersort
    - Particularly where comparisons are cheap and sentinels are useable

Thanks for listening!

Results 5: Time as function of presortedness

